

DEZVOLTAREA UNUI SISTEM DE CONTROL DE ZBOR ȘI CAPTURĂ DE IMAGINI AERIENE PENTRU DRONE

Candidat: Norbert KONDORA

Coordonator științific: Dr. Habil. Ing. Mihai V. MICEA

Sesiunea: Septembrie 2026

REZUMAT

Lucrarea de față descrie proiectarea, implementarea unui sistem de control de zbor pentru o dronă construită integral de la zero, precum și al unui subsistem dedicat capturii și salvării de imagini aeriene. Arhitectura hardware a sistemului este împărțită pe două plăci de dezvoltare ESP32-S3 astfel: un controler de zbor, responsabil pentru stabilizarea dronei și un modul de telemetrie care asigură interfața de control utilizată de pilot, printr-o pagina web disponibilă prin Wi-Fi, precum și achiziția, procesarea și salvarea pe un card de memorie a imaginilor capturate cu modulul cameră compatibil. Comunicarea între cele două plăci se realizează printr-un protocol serial simplu iar structura mesajelor este implementată într-un modul software partajat între cele două proiecte ESP-IDF corespunzătoare pentru fiecare dintre cele două module de dezvoltare.

CUPRINS

1	Introducere.....	4
1.1	Contextul temei alese.....	4
1.2	Motivația și scopul proiectului.....	4
1.3	Soluția propusă.....	4
1.4	Contribuția autorului.....	5
2	Soluții similare.....	6
2.1	Sisteme de pilotare automată de uz profesional.....	6
2.2	Firmware pentru drone de mici dimensiuni.....	6
2.3	Soluții bazate pe microcontrolere ESP32.....	7
3	Fundamente teoretice.....	8
3.1	Principiul de zbor al unei drone cu patru motoare.....	8
3.1.1	Subactuarea.....	10
3.1.2	Pragul minim al motoarelor.....	10
3.2	Reglarea automată prin regulator PID.....	11
3.2.1	Componenta proporțională.....	11
3.2.2	Componenta integrală.....	12
3.2.3	Componenta derivativă.....	13
3.2.4	Saturarea și acumularea excesivă.....	15

1 INTRODUCERE

1.1 CONTEXTUL TEMEI ALESE

Dronele au devenit în ultimii ani instrumente obișnuite într-o multitudine de domenii: cartografiere, inspectia cladirilor și a instalatiilor industriale, agricultură, filmare și fotografiere aeriană. Majoritatea acestor aplicații se bazează însă pe un lucru pe care îl luăm ca atare anume, semnalul GPS folosit pentru pozitionarea dronei.

În interiorul cladirilor, în apropierea structurilor metalice sau în spații închise, nu ne putem baza pe acest semnal deoarece este imprecis în condițiile de menționate mai sus. Zborul în astfel de condiții rămâne o problemă distinctă și dificilă, care presupune alte metode pentru determinarea și menținerea poziției.[1] Dronele de dimensiuni mici, capabile să zboare în spații închise, formează astfel o categorie aparte, cu cerințe proprii atât la nivel hardware, cât și la nivel software.

1.2 MOTIVAȚIA ȘI SCOPUL PROIECTULUI

Motivația acestei lucrări a pornit din dorința de a înțelege, cât mai profund, cum funcționează un sistem care menține echilibrul și poziția unei drone și de a parcurge personal tot procesul de dezvoltare.

La aceasta se adaugă un obiectiv concret: capturarea de imagini aeriene destinate reconstrucției tridimensionale a unui obiect sau a unei încăperi. Această utilizare impune cerințe diferite de cele ale unei drone obișnuite cu capacități de fotografiere/filmare. Imaginile trebuie realizate în condiții de iluminare cât mai constante între cadrele obținute, altfel programele de reconstrucție interpretează variațiile de luminozitate ca modificări ale scenei reale iar rezultatul se degradează.[2]

Scopul lucrării este, prin urmare, proiectarea, implementarea și testarea unei drone complete, construite de la zero, folosind componente ieftine și larg disponibile.

1.3 SOLUȚIA PROPUȘĂ

Soluția propusă are ca element central o alegere arhitecturală: drona folosește două microcontrolere de mici dimensiuni în loc de unul singur.

Primul se ocupă exclusiv de stabilizarea dronei. Citește permanent date de la senzori și reglează turatia celor patru motoare. Al doilea se ocupă de restul sarcinilor: comunicatia cu utilizatorul, camera și salvarea imaginilor pe un card de memorie. Această separare a sarcinilor a fost aleasă cu scopul de a preveni posibile perturbări ale sistemului de control cauzate de întâzieri în transmiterea datelor sau operații lente precum salvarea unei fotografii pe cardul de memorie.

Comanda dronei se face printr-o rețea WiFi, creată de unul dintre cele două microcontrolere, folosind o pagină web accesibilă dintr-un browser obișnuit. Acestă

alegere elimină nevoia unei telecomenzi specializate și face drona utilizabilă cu dispozitive pe care oricine le are deja.

În paralel cu programul de pe dronă a fost realizată o aplicație separată, cu rol de depanare și validare folosită pentru ajustarea comportamentului dronei, precum și pentru înregistrarea și analiza ulterioară a zborurilor.

1.4 CONTRIBUȚIA AUTORULUI

Contribuțiile proprii ale acestei lucrări, asumate de autor, sunt următoarele:

- Scrierea logicii de control, integrarea componentelor software necesare senzorilor, parametrizarea și verificarea experimentală a sistemului.
- Arhitectura cu două microcontrolere separate, cu izolarea completă a funcției de zbor față de restul sistemului, reprezintă o abordare diferită de cea a soluțiilor comparabile analizate în capitolul următor, care execută toate funcțiile pe un singur procesor.
- Comanda prin pagină web, fără un echipament radio dedicat, face drona utilizabilă fără achiziția unei telecomenzi specializate.
- Subsistemul de capturare a imaginilor.
- Instrumentul de analiză dezvoltat în paralel a permis identificarea, pe baza datelor înregistrate în timpul zborului, a unor probleme reale de comportament ale sistemului de control, care altfel ar fi rămas invizibile.

Rezultatul nu este un produs finit. O parte dintre parametrii sistemului de control sunt încă valori estimate, iar câteva verificări practice rămân de efectuat. Lucrarea prezintă onest atât ceea ce funcționează, cât și problemele identificate și rămase deschise, considerând că modul în care acestea au fost descoperite și analizate constituie el însuși un rezultat al proiectului.

2 SOLUȚII SIMILARE

Dezvoltarea unui sistem de control de zbor propriu se raportează la un ecosistem matur de soluții existente, care acoperă un spectru larg, de la sisteme de pilotare automată de uz profesional până la platforme de cercetare de dimensiuni reduse și proiecte experimentale bazate pe microcontrolere de uz general.

2.1 SISTEME DE PILOTARE AUTOMATĂ DE UZ PROFESIONAL

Platformele PX4[3] și ArduPilot[4] reprezintă standardul în domeniul sistemelor de pilotare automată open source, fiind utilizate atât în cercetare, cât și în aplicații comerciale. Ambele oferă funcționalități complete de navigație autonomă, planificare de misiune, gestiune a modurilor de zbor și redundanță senzorială, inclusiv suport pentru menținerea poziției în interior pe baza fuziunii dintre un senzor de flux optic și unul de măsurare al distanței.

Aceste sisteme sunt însă proiectate pentru a rula pe platforme hardware dedicate, din familia Pixhawk, construite în jurul unor microcontrolere de clasă superioară și echipate cu unități inerțiale redundante [5]. Volumul codului sursă, cerințele de memorie și complexitatea arhitecturală fac ca aceste soluții să nu fie transferabile pe o placă de dezvoltare de dimensiunea și costul celor utilizate în prezentul proiect.

2.2 FIRMWARE PENTRU DRONE DE MICI DIMENSIUNI

Pentru dronele de mici dimensiuni, categorie în care se încadrează și platforma dezvoltată, soluția dominantă este firmware-ul Betaflight [6]. Acesta este optimizat pentru performanță de zbor manual, cu accent pe latență redusă și pe calitatea răspunsului buclilor de reglare a vitezei unghiulare, și rulează pe controlere de zbor dedicate, construite în jurul microcontrolerelor din familia STM32.

Diferențele față de soluția propusă sunt structurale. Betaflight presupune existența unei legături radio dedicate, cu emițător și receptor specializate, comanda fiind transmisă prin protocoale de radiocomandă consacrate. Subsistemul video, atunci când există, este independent de firmware-ul de zbor: imaginea este transmisă analogic sau digital către ochelarii pilotului, fără a fi procesată sau stocată la bordul aeronavei. Nu există, în consecință, nicio integrare între funcția de zbor și funcția de achiziție de imagini, acestea fiind două sisteme fizic separate care coexistă pe aceeași structură mecanică.

2.3 SOLUȚII BAZATE PE MICROCONTROLERE ESP32

Proiectul ESP-Drone, dezvoltat de Espressif, este cea mai apropiată soluție de tema lucrării: un firmware de zbor pentru microcontrolere din familiile ESP32, ESP32-S2 și ESP32-S3, portat din firmware-ul platformei Crazyflie [7]. Soluția include comandă prin Wi-Fi, dintr-o aplicație mobilă dedicată, iar prin modulul opțional de extensie pentru menținerea poziției utilizează aceeași pereche de senzori: PMW3901 și VL53L1X. Dată fiind această suprapunere, adaptarea acestui proiect la platforma hardware proprie a fost evaluată explicit, ca alternativă la dezvoltarea unui firmware propriu. Analiza a identificat câteva obstacole concrete:

- **Mecanismul bazat pe monitorizarea tensiunii acumulatorului** - ESP-Drone folosește tensiunea acumulatorului atât în lanțul de distribuție a puterii, cât și în mecanismul de supraveghere a stării aeronavei, o funcționalitate care nu era disponibilă sistemului discutat în acest proiect la momentul evaluării.
- **Incompatibilitatea arhitecturală** - ESP-Drone folosește un protocol de comunicație diferit, bazat pe Wi-Fi și care este utilizată împreună cu o aplicație mobilă dedicată. Modulul de telemetrie, tabloul de bord web, protocolul serial dintre cele două plăci, driverul pentru unitatea de măsură inerțială și instrumentul de depanare, dezvoltate în cadrul acestei lucrări, nu au nimic în comun cu acea arhitectură.
- **Coeficienți de reglare ale buclelor PID diferite** - Coeficienții de reglare sunt calibrați pentru structura mecanică diferită, deci ar fi fost necesară oricum o reglare completă, iar dificultățile identificate în timpul testărilor, descrise în capitolul dedicat rezultatelor, provin din parametri specifici acestei construcții, pentru care niciun firmware existent nu oferă valori.

Proiectul a fost folosit, în schimb, ca implementare de referință: modul în care măsurătoarea de flux optic este corectată cu viteza unghiulară măsurată de giroscop, precum și configurarea telemetrului laser, au fost comparate cu soluția adoptată acolo.

3 FUNDAMENTE TEORETICE

În acest capitol sunt prezentate noțiunile teoretice pe care se bazează sistemul realizat, împreună cu uneltele și bibliotecile folosite. Pentru fiecare noțiune este prezentat și modul concret în care aceasta apare implementată.

3.1 PRINCIPIUL DE ZBOR AL UNEI DRONE CU PATRU MOTOARE

La baza funcționării unei drone se află Principiul al 3-lea al Mecanicii lui Newton: pentru fiecare acțiune, există o reacție egală și de sens opus. Mai exact, pe măsură ce elicele dronei împing aerul în jos, ele creează o forță de portanță în sus. În consecință, această forță de portanță permite dronelor să plutească, să urce sau să coboare cu o precizie remarcabilă. Mai mult, prin reglarea vitezei elicelor, dronele își pot controla mișcările și își pot menține stabilitatea în timpul zborului.[8]

Pentru a înclina drona într-o parte, motoarele de pe acea parte trebuie să producă mai puțină forță decât cele de pe partea opusă. Diferența dintre forțe creează un moment care rotește drona iar, ca urmare a înclinării, aceasta se deplasează pe axa de translație corespunzătoare. La fel se obține și înclinarea față-spate. Rotația în jurul axei verticale funcționează pe alt principiu. Când o elice se rotește într-un sens, apare un moment de reacție care acționează asupra dronei în sensul opus. Din acest motiv două dintre elice, aflate pe aceeași diagonală, se rotesc în sens orar și două, aflate pe cealaltă diagonală, se rotesc în sens antiorar, astfel încât în zbor normal aceste forțe se anulează reciproc. Dacă accelerăm perechea care se rotește antiorar și le încetinim pe celelalte două, drona se va roti în sens orar.

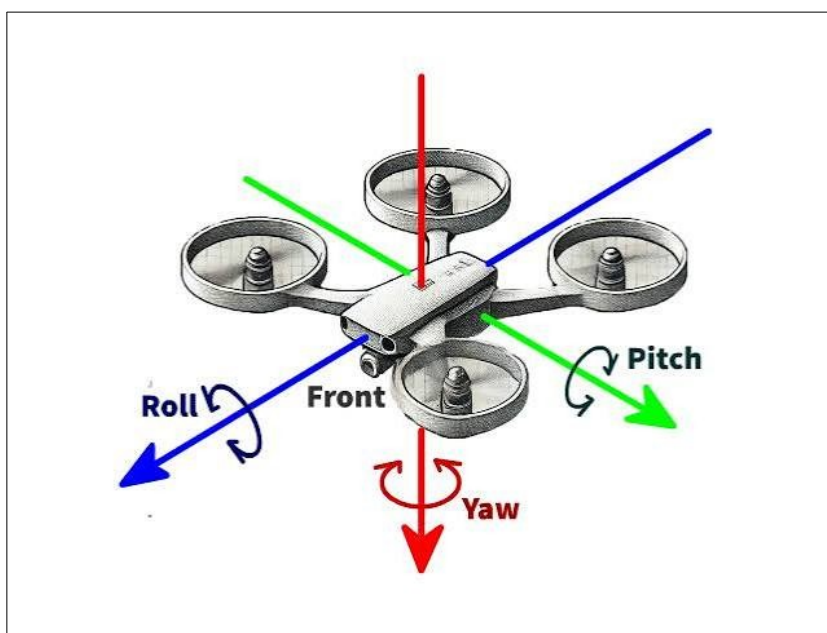


Figura 1 – Principalele mișcări ale unei drone(sursa: [8])

Cele trei comenzi de rotație, ruliu(*roll*), tangaj(*pitch*) și girație(*yaw*) împreună cu comanda de propulsie, trebuie transformate în patru valori individuale, câte una pentru fiecare motor. Această operație este denumită în practică mixare și este implementată astfel:

```
mix[MOTOR_FRONT_LEFT] = roll - pitch - yaw;  
mix[MOTOR_FRONT_RIGHT] = -roll - pitch + yaw;  
mix[MOTOR_REAR_LEFT] = roll + pitch + yaw;  
mix[MOTOR_REAR_RIGHT] = -roll + pitch - yaw;
```

Semnele din fiecare coloană rezultă din poziția motorului și din sensul lui de rotație. Pentru înclinarea laterală, motoarele din stânga primesc semnul plus, iar cele din dreapta semnul minus, astfel încât o comandă pozitivă ridică partea stângă. Pentru înclinarea față-spate, motoarele din spate primesc plus. Pentru rotația pe verticală semnele urmează sensul de rotație al elicei: motoarele care se rotesc antiorar primesc plus.

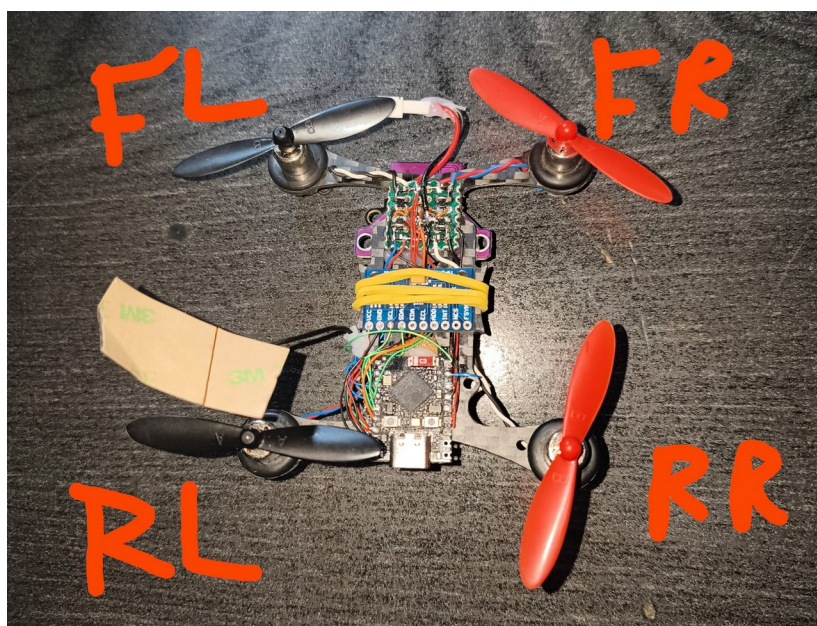


Figura 2 – Poziționarea motoarelor

3.1.1 SUBACTUAREA

Un corp în spațiu are șase grade de libertate: se poate deplasa pe trei axe de translație și pe trei axe de rotație. Drona are însă doar patru comenzi independente, propulsia totală și cele trei momente, *roll*, *pitch* și *yaw*. Se spune că sistemul este subactuat, adică are mai puține comenzi decât grade de libertate.

Consecința este importantă pentru felul în care este organizat programul. Forța produsă de elice este întotdeauna perpendiculară pe corpul dronei, deci, stând dreaptă, drona nu poate produce nicio forță orizontală. Ca să se deplaseze în față, trebuie mai întâi să se încline în față, astfel încât o parte din forța elicelor să acționeze pe orizontală.

Din acest motiv, partea de program care se ocupă de deplasarea orizontală nu comandă direct motoarele, ci calculează un unghi de înclinare, pe care altă parte a programului îl realizează. Această organizare pe niveluri este descrisă în secțiunea 3.3.

3.1.2 PRAGUL MINIM AL MOTOARELOR

Motoarele folosite sunt motoare de curent continuu cu perii. Acestea sunt comandate folosind semnale de tip PWM (*Pulse Width Modulation*). Un astfel de motor comandat de un semnal PWM cu 1% factor de umplere nu se va roti și este posibil ca valoarea factorului de umplere la care motorul începe să se rotească să fie diferită de la motor la motor.

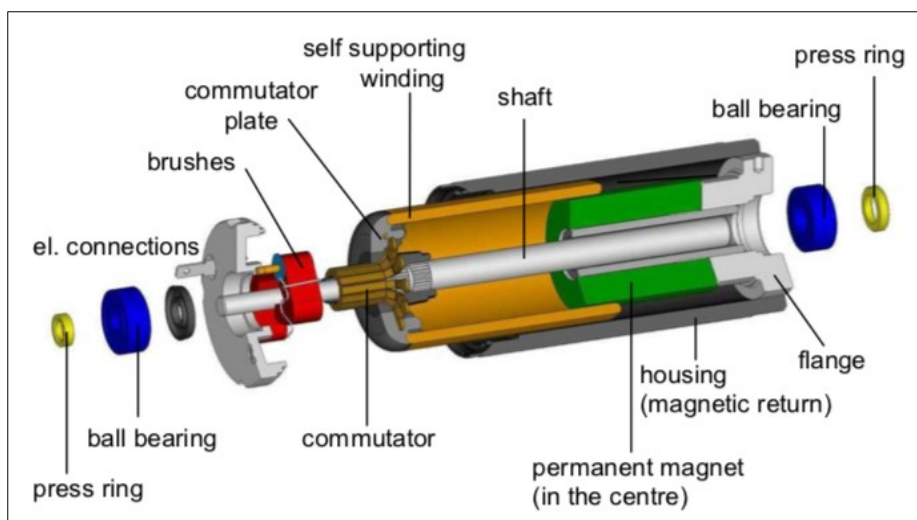


Figura 3 – Motor cu perii de tip *coreless* (sursa: [9])

Funcția de mixare calculează mai întâi doar contribuția de atitudine, fără propulsia totală, ca un set de valori în jurul lui zero. Aceste valori sunt cu semn: o valoare negativă înseamnă că motorul respectiv trebuie să se rotească mai puțin decât media. Comanda finală a fiecărui motor este suma dintre propulsia totală și diferența corespunzătoare din funcția de mixare care poate fi o valoare negativă. Factorul de umplere este limitat la

intervalul 0% -100 % deci o valoare negativă de comandă va rezulta un factor de umplere de 0%, moment în care motorul se oprește, posibil chiar în momentul în care drona ar avea cea mai mare nevoie de corecție. Pentru a evita o astfel de situație, în program este definit un prag minim sub care motoarele nu coboară niciodată cât timp drona este armată.

```
#define MOTOR_IDLE_THRUST 0.06f
```

Această valoare reprezintă un factor de umplere de 6% minim la care toate cele patru motoare se rotesc. Această valoare a fost măsurată experimental, detaliile fiind prezentate în secțiunea de rezultate experimentale.

3.2 REGLAREA AUTOMATĂ PRIN REGULATOR PID

Regulatorul PID este pe departe cel mai comun algoritm de control. Majoritatea buclelor de *feedback* sunt controlate de acest algoritm sau de variații ale acestuia. Este implementată în multe forme diferite, ca regulator independent sau ca un sistem ierarhic distribuit de control al proceselor. Ingineri din toată lumea utilizează acest algoritm în munca lor de zi cu zi.[10]

În cazul nostru, pentru ca drona să-și mențină echilibrul nu este suficient să comandăm motoarele o singură dată. Perturbațiile, un curent de aer, o elice ușor dezechilibrată, o greutate distribuită inegal, influențează tipul comenzii necesare motorului. Aici intervine acest tip de regulator în buclă închisă care masoară orientarea reală, o compară cu cea dorită și corectează comanda în funcție de diferența dintre ele.

Diferența dintre valoarea dorită și cea măsurată se numește eroare:

```
const float error = setpoint - measurement;
```

Regulatorul PID calculează comanda pe baza acestei erori, folosind trei componente: proporțională, integrală și derivativă care la rândul lor utilizează 3 parametri care influențează comportamentul componentei: K_p , K_i , respectiv K_d . În lucrarea citată în acest subcapitol se lucrează cu forma standard a acestor parametri K , T_i și T_d unde T_i și T_d reprezintă constante de timp. Subcapitolele următoare și implementarea de față utilizează forma paralelă a acestor parametri, K_p , K_i și K_d independenți, legătura fiind $T_i = K_p/K_i$ și $T_d = K_d/K_p$ [10].

3.2.1 COMPONENTA PROPORȚIONALĂ

Componenta proporțională generează o comandă direct proporțională cu eroarea curentă:

```
const float p_term = pid->kp * error;
```

Coeficientul de proporționalitate K_p stabilește cum reacționează regulatorul: la o abatere de un grad față de unghiul dorit, comanda produsă este de o mie de ori mai mare pentru un coeficient de o mie decât pentru unul unitar.

Limitarea fundamentală a componentei proporționale este că nu poate anula complet abaterea. Rezultatul este o diferență permanentă între valoarea dorită și cea măsurată, numită eroare staționară (*steady-state error*) [10]. Aceasta apare din cauza modului în care funcționează componenta: comanda de corecție există doar pentru că există eroare. Dacă eroarea ar fi zero, regulatorul nu ar avea ce să corecteze. În prezența unei perturbații permanente, o baterie montată ușor descentrat, un motor care se rotește mai încet decât celelalte la aceeași comandă, bucla va rămâne într-un punct de echilibru în care eroarea are exact valoarea necesară pentru a genera comanda ce compensează perturbația.

Mărimea acestei erori depinde de coeficientul de proporționalitate K_p : cu cât acesta este mai mare, cu atât abaterea necesară pentru a produce aceeași comandă este mai mică. Eroarea staționară scade, prin urmare, odată cu creșterea coeficientului, dar nu se anulează niciodată, iar dincolo de o anumită valoare bucla va oscila și, în cele din urmă, va deveni instabilă.

Practic, doar cu componenta proporțională, drona ar zbura permanent ușor înclinată, iar înclinarea ar fi cu atât mai mică cu cât reglajul este mai agresiv.

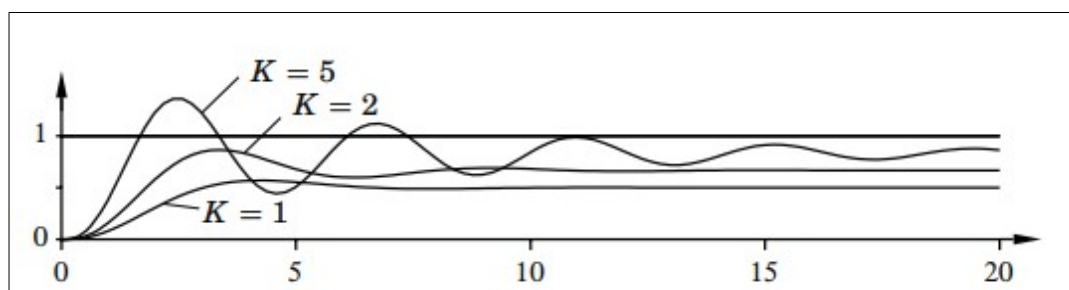


Figura 4 – Simularea unui sistem în buclă închisă folosind doar componenta proporțională. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru valori diferite ale K_p . (sursa: [10])

3.2.2 COMPONENTA INTEGRALĂ

Componenta integrală acumulează eroarea în timp și adaugă rezultatul la comandă. Rolul ei este acela de a garanta că, în regim staționar, mărimea reglată ajunge efectiv la valoarea dorită.

Modul prin care funcționează poate fi înțeles printr-un raționament simplu. Să presupunem că sistemul ar ajunge într-un regim staționar în care eroarea rămâne constantă și diferită de zero. Componenta integrală ar continua atunci să acumuleze, iar comanda ar crește la nesfârșit, ceea ce contrazice ipoteza de regim staționar, în care comanda trebuie să fie constantă. Singura situație în care un regim staționar este posibil

este, prin urmare, aceea în care eroarea este nulă. Prezența componentei integrale elimină astfel eroarea staționară descrisă anterior [10].

Din punct de vedere istoric, această componentă a purtat numele de „resetare automată”: înainte de apariția ei, compensarea unei perturbații permanente se făcea prin ajustarea manuală a unei valori de bază adăugate comenzii, operație care trebuia repetată la fiecare schimbare a condițiilor de funcționare [10].

În cazul dronei, componenta integrală este cea care compensează un dezechilibru mecanic permanent: dacă centrul de greutate nu coincide cu centrul geometric, ea acumulează treptat până când diferența de turație dintre motoare anulează efectul, iar drona zboară orizontal fără ca pilotul să corecteze continuu.

Componenta integrală are însă și dezavantaje. Ea introduce o întârziere suplimentară în răspunsul buclei, reducând stabilitatea, și prezintă fenomenul de acumulare excesivă descris în secțiunea următoare.

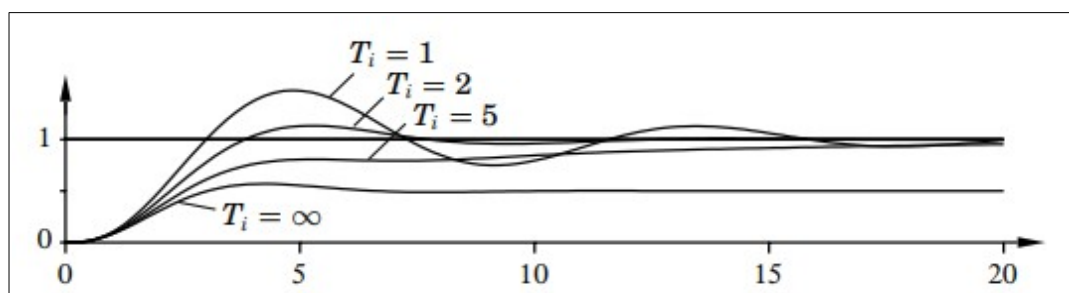


Figura 5 - Simularea unui sistem în buclă închisă folosind componenta proporțională și cea integrală. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru $K_p = 1$ și pentru diferite valori ale K_i unde $K_i = K_p/T_i$. (sursa: [10])

3.2.3 COMPONENTA DERIVATIVĂ

Componenta derivativă este proporțională cu viteza de variație a mărimii reglate. Rolul ei este îmbunătățirea stabilității buclei.

Necesitatea ei constă în faptul că orice sistem fizic răspunde cu întârziere: între momentul în care se modifică turația motoarelor și momentul în care drona își schimbă efectiv orientarea trece un interval de timp. Un regulator care se raportează exclusiv la eroarea prezentă va continua să comande corecție și după ce mișcarea de revenire a început deja, ceea ce produce depășirea valorii dorite și, în consecință, oscilație.

Un regulator proporțional derivativ este, din acest punct de vedere, echivalent cu un regulator proporțional care nu acționează asupra erorii curente, ci asupra erorii anticipate cu T_d secunde în avans, unde $T_d = K_d / K_p$. Constanta T_d reprezintă astfel orizontul de predicție [10].

Practic, dacă drona este înclinată la trei grade, dar se rotește deja rapid înapoi spre orizontală, componenta derivativă reduce corecția, pentru că mișcarea de revenire este deja în curs și o corecție suplimentară ar produce depășire în sens opus.

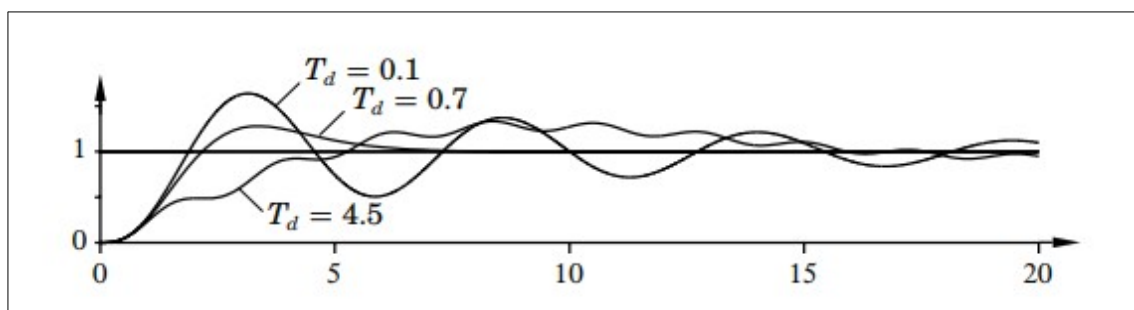


Figura 6 - Simularea unui sistem în buclă închisă folosind componentele proporțională, integrală și derivativă. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru $K_p = 3$, $K_i = 3$ și diferite valori pentru K_d unde $K_d = K_p \times T_d$ (sursa: [10])

Componenta derivativă ridică însă două probleme practice, ambele cu soluții consacrate. Prima apare la modificarea valorii dorite. În situația în care o comandă se modifică brusc la fel și eroarea se va modifica brusc. Rezultatul este o comandă care nu corespunde niciunei mișcări reale a procesului. Soluția uzuală este calcularea derivatei pe baza mărimii măsurate, nu a erorii. Întrucât diferența dintre cele două este tocmai derivata referinței, eliminarea acestui termen face ca regulatorul să reacționeze exclusiv la evoluția reală a procesului, nu la comenzile operatorului [10].

A doua problemă este amplificarea zgomotului. Caracteristica de frecvență a operatorului de derivare crește proporțional cu frecvența, ceea ce înseamnă că un zgomot de amplitudine mică, dar de frecvență ridicată, inerent oricărei măsurători, cu atât mai mult uneia provenite de la un senzor montat pe o structură care vibrează, produce o contribuție considerabilă în comandă. Din acest motiv componenta derivativă nu se implementează niciodată în formă pură, ci întotdeauna însoțită de o filtrare trece-jos, a cărei frecvență de tăiere stabilește un compromis între atenuarea zgomotului și întârzierea introdusă în răspuns [10].

Modul concret în care aceste două soluții au fost implementate, împreună cu valorile alese pentru fiecare buclă, este prezentat în capitolul următor.

3.2.4 SATURAREA ȘI ACUMULAREA EXCESIVĂ

Orice actuator are limitări. În cazul de față, viteza la care se pot roti motoarele utilizate este limitată. În cazul în care regulatorul cere mai mult decât pot motoarele livra, sistemul intră în saturație iar bucla de control se deschide: oricât ar crește comanda calculată, motoarele nu vor mai răspunde suplimentar.

Problema apare la componenta integrală. Aceasta nu are cum să știe că sistemul este saturat și continuă să adune eroarea, care nu scade. Valoarea acumulată poate crește foarte mult, iar când situația se normalizează ea trebuie mai întâi consumată, timp în care comanda rămâne la maxim deși nu mai este nevoie. Fenomenul poartă numele de *integrator windup*.

O soluție care previne acest fenomen constă în integrarea condiționată. Înainte de a accepta pasul de integrare, programul va calcula ce comandă ar rezulta și numai dacă eroarea curentă ar împinge comanda și mai departe în saturație aceasta nu se mai acumulează. Modul concret de implementare va fi prezentat în capitolul următor.

3.3 STRUCTURA DE REGLARE ÎN CASCADĂ

Sistemul nu folosește un singur regulator, ci mai multe, organizate pe trei niveluri. Fiecare nivel stabilește valoarea dorită pentru nivelul de sub el: nivelul exterior calculează, din altitudine și din viteza de deplasare, comanda de propulsie și unghiurile de înclinare dorite; nivelul intermediar transformă unghiul dorit în viteză unghiulară dorită; nivelul interior transformă viteza unghiulară dorită în comenzi către motoare.

Motivul acestei organizări a fost explicat în secțiunea privind subactuarea: drona nu poate comanda direct deplasarea, ci doar înclinarea.

Frecvențele de execuție diferă pentru că mărimile se schimbă cu viteze diferite. Viteza unghiulară se poate modifica foarte repede și este măsurată de giroscop la fiecare ciclu, deci bucla interioară rulează cel mai des. Unghiul se schimbă mai lent. Altitudinea și viteza de deplasare se schimbă cel mai lent și, în plus, senzorii care le măsoară furnizează date mai rar: telemetrul laser în jur de 33 de măsurători pe secundă.

În program, cele trei bucle nu rulează ca sarcini separate, ci sunt derivate dintr-un singur ciclu de 1000 Hz, prin numărare:

```
#define INNER_LOOP_HZ      1000
#define MID_LOOP_DIVIDER   4      // 1000 / 4 = 250 Hz
#define OUTER_LOOP_DIVIDER 20     // 1000 / 20 = 50 Hz
```

Avantajul este că cele trei bucle nu se pot defaza una față de alta. Dacă ar fi sarcini separate, întârzierea dintre calculul unei valori dorite și folosirea ei ar varia de la un ciclu la altul, ceea ce ar face comportamentul mai greu de analizat.

Regula uzuală pentru astfel de structuri este ca fiecare buclă interioară să ruleze de cel puțin cinci ori mai des decât cea exterioară, astfel încât, din punctul de vedere al buclei exterioare, cea interioară să pară instantanee [11]. În implementarea realizată raportul dintre nivelul exterior și cel intermediar este 5, deci respectă recomandarea; între nivelul intermediar și cel interior este 4, ușor sub valoarea recomandată. O consecință practică a acestei structuri este ordinea în care trebuie făcută reglarea: dinspre interior spre exterior. Fiecare buclă exterioară presupune că bucla de sub ea funcționează deja corect, deci nu are sens să fie reglată înaintea acesteia.

3.4 ESTIMAREA ORIENTĂRII PRIN FUZIUNEA SENZORILOR

Pentru a corecta orientarea, sistemul trebuie mai întâi să o cunoască. Drona nu dispune de un senzor care să indice direct unghiul de înclinare, ci de un accelerometru și un giroscop, iar unghiul trebuie calculat din datele furnizate de acestea.

3.4.1 ACCELEROMETRUL

Accelerometrul măsoară accelerația care acționează asupra lui. Este un dispozitiv electromecanic care detectează fie forțe statice, fie forțe dinamice de accelerație. Forțele statice includ gravitația, în timp ce forțele dinamice pot include vibrații și mișcare [12].

Această proprietate este utilă. Când drona stă nemișcată sau se deplasează uniform, singurul lucru pe care îl măsoară accelerometrul este gravitația. Dacă drona este perfect orizontală, toată accelerația apare pe axa verticală; dacă este înclinată, o parte din ea se regăsește pe celelalte axe, iar din raportul dintre componente se poate calcula unghiul [13]:

```
*roll_angle = (atan2(acc_y_g, acc_z_g) * (180.0 / M_PI)) - roll_offset;  
*pitch_angle = (atan2(-acc_x_g, sqrt(acc_y_g * acc_y_g + acc_z_g * acc_z_g))  
                * (180.0 / M_PI)) - pitch_offset;
```

Valorile scăzute la final sunt corecții măsurate la pornire, cu drona așezată orizontal, pentru a compensa faptul că senzorul nu este montat perfect drept pe placă.

Accelerometrul are însă două probleme. Prima este că nu poate distinge între înclinare și accelerație: dacă drona accelerează lateral fără să se încline, senzorul indică practic același lucru ca în cazul unei înclinări. A doua este că vibrațiile produse de elice îl afectează puternic.

3.4.2 GIROSCOPUL

Giroscopul măsoară o altă mărime decât accelerometrul: viteza de rotație, exprimată în grade pe secundă. Nu are nicio legătură cu gravitația și nu este afectat de accelerațiile liniare.

Obținerea unghiului se face prin integrare: la fiecare ciclu, unghiul crește cu viteza măsurată înmulțită cu timpul scurs de la ciclul anterior. Aici apare însă problema fundamentală a acestui senzor, integrarea nu acumulează doar semnalul util, ci și erorile de măsurare.

Două tipuri de eroare contează, iar ele se acumulează diferit. Primul este *offsetul* constant: senzorul indică o valoare mică, diferită de zero, chiar și atunci când stă perfect nemișcat. Integrat, acest *offset* produce o eroare de unghi care crește liniar cu timpul. O abatere de doar 0,5 grade pe secundă devine, după un minut de zbor, o eroare de 30 de grade. Al doilea este zgomotul aleator, provenit din agitația termică a structurii mecanice a senzorului. Integrat, acesta produce o deplasare aleatoare a unghiului estimat, a cărei abatere crește proporțional cu rădăcina pătrată a timpului. Efectul este mai lent decât cel al *offsetului*, dar nu poate fi eliminat prin calibrare, fiind aleator prin natura lui [14].

La acestea se adaugă faptul că *offsetul* însuși nu este strict constant, ci variază lent cu temperatura și în timp, ceea ce produce tot o eroare cu creștere liniară. Contramăsura uzuală este determinarea *offsetului* la pornire, cu senzorul nemișcat: se măsoară valoarea indicată timp de câteva sute de eșantioane, se calculează media, iar aceasta se scade din toate citirile ulterioare. Procedura elimină componenta constantă existentă la momentul pornirii, dar nu rezolvă variația ulterioară cu temperatura și nu are niciun efect asupra zgomotului aleator [14].

Concluzia practică este că un giroscop nu poate furniza singur orientarea absolută a unei aeronave pe durata unui zbor. Oricât de bine ar fi calibrat, estimarea obținută prin integrare se îndepărtează treptat de realitate și nu se poate corecta singură, întrucât nu dispune de nicio referință externă față de care să se compare.

Astfel cele două surse sunt complementare: accelerometrul este corect pe termen lung dar zgomotos, giroscopul este precis pe termen scurt dar derivează. Modul în care sunt combinate este prezentat în capitolul 4.

3.5 MĂSURAREA ALTITUDINII ȘI A VITEZEI DE DEPLASARE

3.5.1 TELEMETRUL LASER

Pentru măsurarea altitudinii este folosit un senzor ToF (Time of Flight) VL53L1X [15], orientat în jos. Acesta emite un impuls luminos infraroșu și măsoară timpul până când lumina se întoarce, reflectată de podea. Cunoșcând viteza luminii, distanța rezultă direct din acest timp.

Măsurătoarea este considerată validă doar în anumite condiții, verificate în program: distanța trebuie să fie cuprinsă între 3 cm și 1,30 m, iar drona nu trebuie să fie înclinată prea mult, pentru că altfel fasciculul nu mai lovește podeaua de sub senzor, ci o zonă laterală, aflată la altă distanță.

3.5.2 SENZORUL DE FLUX OPTIC

Pentru măsurarea vitezei de deplasare pe orizontală este folosit un senzor PMW3901 [16], orientat în jos. El funcționează pe același principiu ca senzorul optic dintr-un mouse de calculator: fotografiază podeaua foarte des și compară imaginile consecutive pentru a determina cu cât s-a deplasat imaginea între ele.

Un aspect esențial este că senzorul nu măsoară o distanță, ci o deplasare unghiulară. El indică cu cât s-a mișcat imaginea în câmpul lui vizual, nu câți centimetri s-a deplasat drona. Aceeași deplasare unghiulară înseamnă câțiva centimetri dacă drona este aproape de sol și considerabil mai mult dacă este sus.

Din acest motiv, pentru a obține viteza în metri pe secundă este obligatoriu să fie cunoscută și altitudinea [17]:

```
const float flow_x_rad = flow->delta_x / FLOW_COUNTS_PER_RAD;  
...  
const float raw_velocity_x = (translation_x_rad / dt) * state.altitude;
```

Aceasta este și explicația pentru care cei doi senzori nu sunt interschimbabili și niciunul nu poate fi eliminat: fără telemetru nu există altitudine, iar fără altitudine măsurătoarea de flux optic nu poate fi convertită în viteză.

Înainte de calcularea vitezei, programul mai face o corecție. Dacă drona se înclină fără să se deplaseze, imaginea de sub ea se mișcă oricum, iar senzorul ar raporta o deplasare care nu a avut loc. Din măsurătoarea de flux se scade, de aceea, componenta datorată rotației, măsurată de giroscop.

3.6 EXECUȚIA PROGRAMULUI ÎN TIMP REAL

Toate calculele descrise anterior au sens doar dacă sunt executate precis din punctul de vedere la ordini efectuării sarcinilor, priorității sarcinilor și a momentului în care se execută. Un regulator care ar trebui să ruleze la fiecare milisecundă, dar rulează uneori la două milisecunde, calculează greșit componentele integrală și derivativă, pentru că ambele depind de timpul scurs.

Pentru gestionarea acestui aspect, programul folosește sistemul de operare în timp real FreeRTOS, inclus în *framework*-ul ESP-IDF [18]. Acesta permite împărțirea programului în sarcini independente, fiecare cu propria prioritate, și garantează că o sarcină cu prioritate mare o întrerupe pe una cu prioritate mai mică.

Microcontrolerul ESP32-S3 dispune de două nuclee de procesare identice denumite Core 0, deseori referit ca și PRO_CPU, și Core1, deseori referit ca și APP_CPU. Aceste denumiri există cu scopul de a reflecta cum utilizează o aplicație ESP-IDF tipică cele două nuclee. Sarcinile responsabile cu utilizarea protocoalelor de comunicație precum WiFi sau Bluetooth rulează pe Core 0 iar restul sarcinilor rulează pe Core1 cu toate că o sarcină poate fi fixată pe oricare dintre cele două nuclee [18].

În implementarea realizată, bucla de control rulează pe nucleul 1, cu cea mai mare prioritate, iar comunicația cu modulul de telemetrie pe nucleul 0. Astfel, o întârziere apărută în comunicație nu poate întârzia bucla de control.

Din același motiv, în bucla de control nu se execută operații a căror durată nu poate fi garantată: fără citiri de pe rețea, fără așteptare după alte sarcini.

Atunci când două sarcini aflate pe nuclee diferite folosesc aceleași date, de exemplu comenzile primite de la pilot, scrise de sarcina de comunicație și citite de bucla de control, accesul este protejat, astfel încât bucla de control să nu poată citi jumătate din comanda veche și jumătate din cea nouă.

3.7 MEDIUL DE DEZVOLTARE ȘI BIBLIOTECILE UTILIZATE

Firmware-ul ambelor plăci este dezvoltat folosind ESP-IDF (Espressif IoT Development Framework), mediul oficial de dezvoltare pus la dispoziție de Espressif pentru microcontrolerele din familia ESP32. Acesta oferă un lanț de instrumente, API-uri, componente și fluxuri de lucru pentru dezvoltarea de aplicații pentru ESP32-S3 utilizând sistemele de operare Windows, Linux și macOS [19].

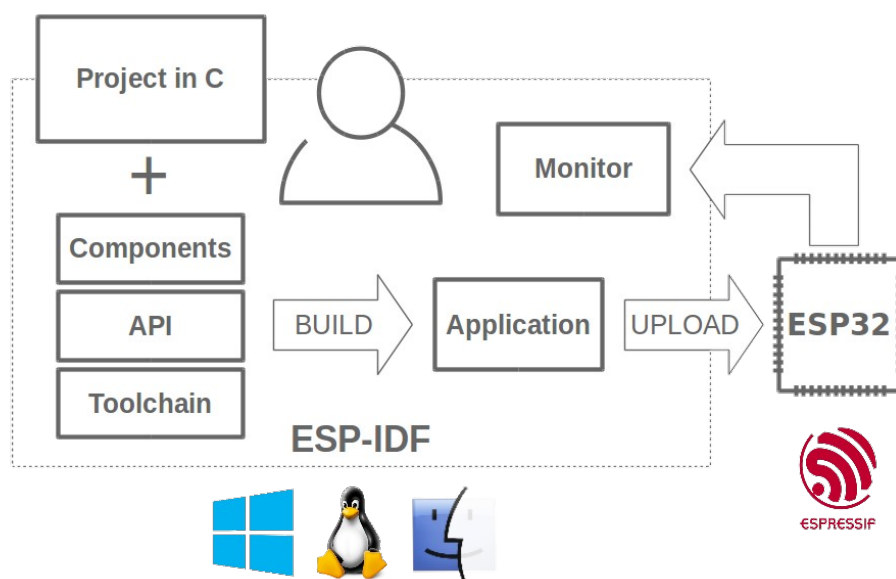


Figura 7 – Funcționarea mediul de lucru ESP-IDF (sursa: [19])

Un proiect ce utilizează acest mediu de dezvoltare este organizat în componente. O componentă este un director care conține codul sursă, fișierele antet publice și un fișier de configurare în care se declară de ce alte componente depinde. Compilarea este gestionată prin CMake, iar sistemul determină automat ordinea de compilare pe baza dependențelor declarate.

Această organizare a fost utilă pentru proiect: fiecare funcție majoră a devenit o componentă separată, driverul senzorului inerțial, estimatorul de orientare, estimatorul de navigație, regulatorul PID, bucla de control, driverul de motoare, monitorizarea acumulatorului, cu limite clare între ele. O componentă poate folosi altă componentă doar dacă a declarat explicit dependența, ceea ce împiedică apariția legăturilor accidentale între părți care ar trebui să rămână independente.

Pe lângă componentele scrise în cadrul proiectului au fost folosite și biblioteci externe. Pentru telemetrul laser este utilizat driverul oficial pus la dispoziție de producător, integrat printr-un strat de adaptare scris în cadrul proiectului, care implementează funcțiile de comunicație pe magistrala I2C. Pentru senzorul de flux optic, secvența de inițializare a registrelor este preluată din driverul cu sursă deschisă publicat de Bitcraze, cu atribuirea și licența păstrate în codul sursă. Modulul de cameră folosește driverul esp32-camera, pus la dispoziție de Espressif. La acestea se adaugă driverele incluse în ESP-IDF pentru perifericele microcontrolerului: magistralele I2C și SPI, interfața serială, generatorul de PWM folosit pentru comanda motoarelor și convertorul analog-digital.

Cele două plăci constituie două proiecte ESP-IDF separate, care partajează o singură componentă comună, cea care definește structura mesajelor schimbate între ele.

Componenta este inclusă în ambele proiecte, astfel încât o modificare a formatului mesajelor se aplică automat de ambele părți și nu pot apărea nepotriviri între ceea ce trimite o placă și ceea ce așteaptă cealaltă.